Value function Approximation Incremental Methods

Master Degree in Control Systems Engineering 2022-2023
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

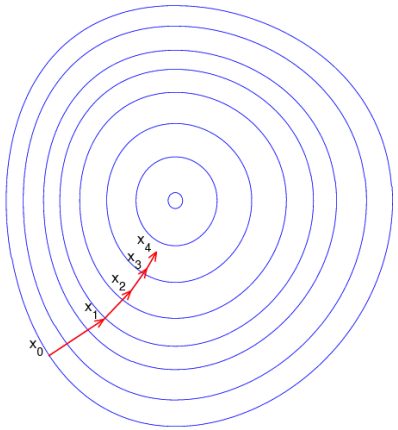
Today's lecture…

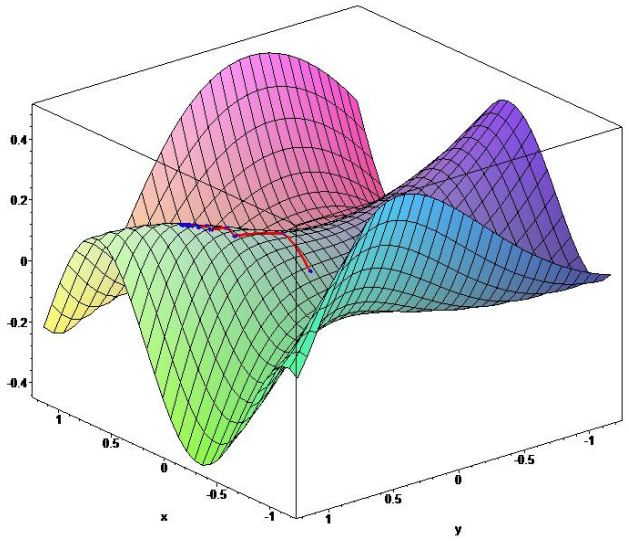
- Introduction to the problem
- Incremental methods (for model-free prediction)
- Batch methods

Gradient Descent

- $J(\mathbf{w})$ *differentiable function* of parameter vector $\mathbf{w}$
- *Gradient* of $J(\mathbf{w})$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$

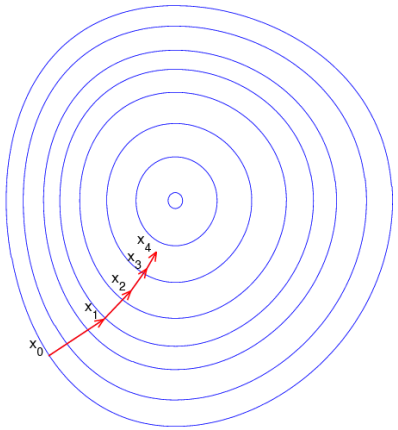




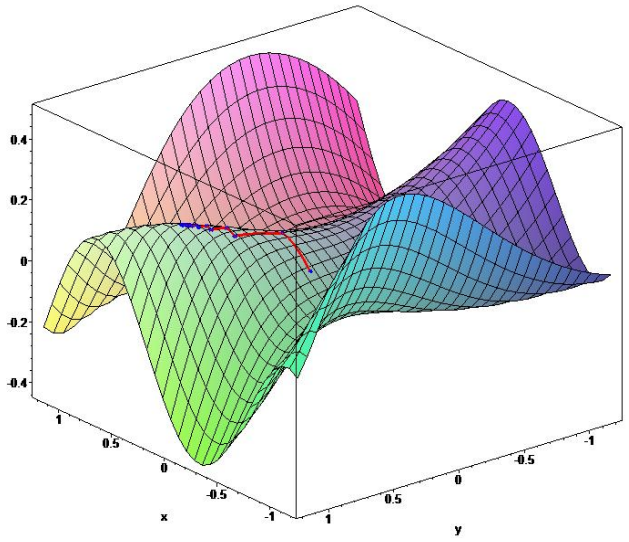
Gradient Descent

- $J(\mathbf{w})$ *differentiable function* of parameter vector $\mathbf{w}$
- *Gradient* of $J(\mathbf{w})$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$



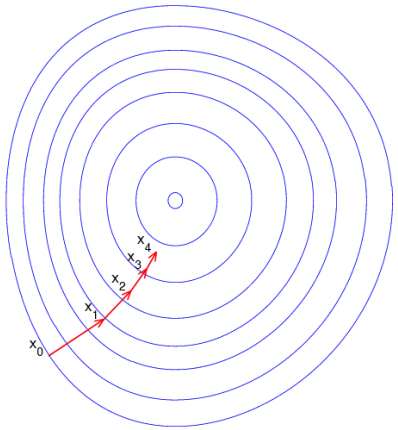
- **Goal** : finding a local minimum of $J(\mathbf{w})$



Gradient Descent

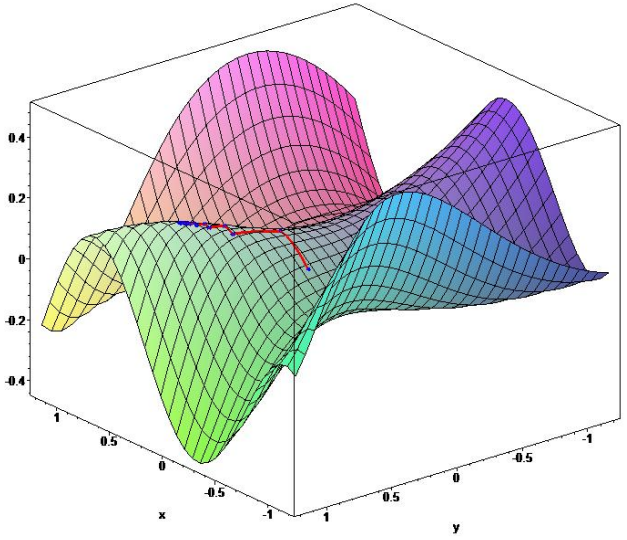
- $J(\mathbf{w})$ *differentiable function* of parameter vector $\mathbf{w}$
- *Gradient* of $J(\mathbf{w})$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$



- **Goal** : finding a local minimum of $J(\mathbf{w})$
- *Update* $\mathbf{w}$ along *negative gradient*

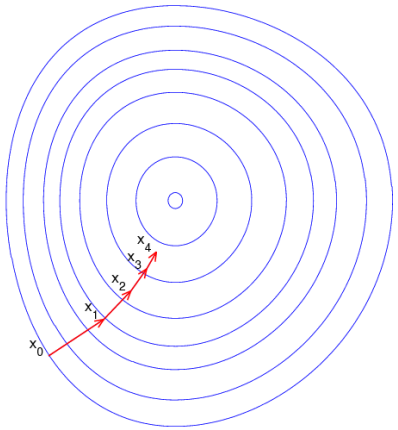
$$\Delta \mathbf{w} = -\frac{1}{2}\alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$



Gradient Descent

- $J(\mathbf{w})$ *differentiable function* of parameter vector $\mathbf{w}$
- *Gradient* of $J(\mathbf{w})$

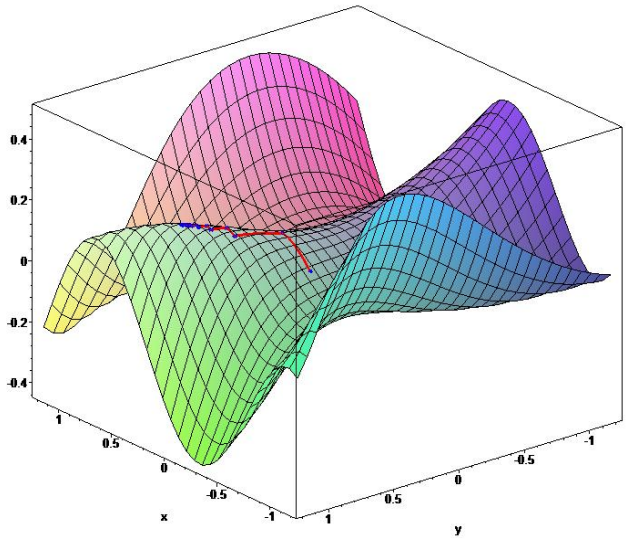
$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$



- **Goal** : finding a local minimum of $J(\mathbf{w})$
- *Update* $\mathbf{w}$ along *negative gradient*

$$\Delta \mathbf{w} = -\frac{1}{2}\alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}$$



Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

…before describing how to apply a stochastic gradient descent approach we discuss a bit the above prediction objective…

The Prediction Objective

Observation : up to now, in the previous lectures, we have not specified an explicit objective for prediction

- in the tabular case the *learned value function could come to equal the true value function* exactly
- *the learned values at each state were decoupled* – an update at one state affected no other

The Prediction Objective

Observation : up to now, in the previous lectures, we have not specified an explicit objective for prediction

- in the tabular case the *learned value function could come to equal the true value function* exactly
- *the learned values at each state were decoupled* – an update at one state affected no other

With approximation, an update at one state affects many others

↳ *not possible to get the values of all states exactly correct!*

The Prediction Objective

Observation : up to now, in the previous lectures, we have not specified an explicit objective for prediction

- in the tabular case the *learned value function could come to equal the true value function* exactly
- *the learned values at each state were decoupled* – an update at one state affected no other

With approximation, an update at one state affects many others

↳ *not possible to get the values of all states exactly correct!*

By assumption more states than weights

↳ *making one state's estimate more accurate might make others' less accurate!*

The Prediction Objective

Observation : up to now, in the previous lectures, we have not specified an explicit objective for prediction

- in the tabular case the *learned value function could come to equal the true value function* exactly
- *the learned values at each state were decoupled* – an update at one state affected no other

With approximation, an update at one state affects many others

↳ *not possible to get the values of all states exactly correct!*

By assumption more states than weights

↳ *making one state's estimate more accurate might make others' less accurate!*

↳ ***We need to say which states we care most about!***

The Prediction Objective

State distribution : $\mu(s) \geq 0, \sum_s \mu(s) = 1$ representing how much we care about the error in each state *s*

The Prediction Objective

State distribution : $\mu(s) \geq 0,\ \sum_s \mu(s) = 1$ representing how much we care about the error in each state *s*

Mean Squared Value Error ($\overline{\text{VE}}$) : it is obtained by weighting the squared error by μ

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \Big[v_\pi(s) - \hat{v}(s, \mathbf{w}) \Big]^2$$

The Prediction Objective

State distribution : $\mu(s) \geq 0,\ \sum_s \mu(s) = 1$ representing how much we care about the error in each state *s*

Mean Squared Value Error ($\overline{\text{VE}}$) : it is obtained by weighting the squared error by μ

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \Big[v_\pi(s) - \hat{v}(s,\mathbf{w}) \Big]^2$$

- Often $\mu(s)$ is chosen to be the fraction of time spent in s
- In the book it is explained how to compute $\mu(s)$ in episodic tasks (*see paragraph in Chapter 9 – The on-policy distribution in episodic tasks*)
- In continuing tasks $\mu(s)$ is the stationary distribution under π

The Prediction Objective

In continuing tasks $\mu(s)$ is the stationary distribution under π

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \Big[v_\pi(s) - \hat{v}(s,\mathbf{w}) \Big]^2$$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

We assume that, for each s, we know the exact value $v_\pi(s)$ (Oracle)

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

We assume that, for each s, we know the exact value $v_\pi(s)$ (Oracle)

Notice that there is generally no $\mathbf{w}$ that gets all the states, or even just the states encountered during the episodes, exactly correct.

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

We assume that, for each s, we know the exact value $v_\pi(s)$ (Oracle)

Notice that there is generally no $\mathbf{w}$ that gets all the states, or even just the states encountered during the episodes, exactly correct.

A good strategy in this case is to minimize error on the observed examples (***Stochastic gradient descent***)

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

- *Gradient descent* finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \, \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}\,(s, \mathbf{w}) \right] \end{aligned}$$

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising *mean-squared error* between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right)^2 \right]$$

- *Gradient descent* finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \, \mathbb{E}_\pi \left[\left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}\,(s, \mathbf{w}) \right] \end{aligned}$$

- *Stochastic gradient descent samples* the gradient

$$\Delta \mathbf{w} = \alpha \, \left(v_\pi(s) - \hat{v}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}\,(s, \mathbf{w})$$

Value function approximation by stochastic gradient descent

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \nabla \mathbf{w}_t$$

$$= \mathbf{w}_t + \alpha \Big[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \Big] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

Stochastic gradient-descent (SGD) methods adjusts the weight vector after each example by *a small amount in the direction that would most reduce the error on that example*

Value function approximation by stochastic gradient descent

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \nabla \mathbf{w}_t$$

$$= \mathbf{w}_t + \alpha \Big[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \Big] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

Stochastic gradient-descent (SGD) methods adjusts the weight vector after each example by *a small amount in the direction that would most reduce the error on that example*

Over many examples, making small steps, the overall effect is to minimize an average performance measure such as the J.

Value function approximation by stochastic gradient descent

Why SGD takes only a small step in the direction of the gradient?

Could we not move all the way in this direction and completely eliminate the error on the example?

Value function approximation by stochastic gradient descent

Why SGD takes only a small step in the direction of the gradient?

Could we not move all the way in this direction and completely eliminate the error on the example?

…we do not seek or expect to find a value function that has zero error for all states, but *only an approximation* that balances the errors in different states…

… if we completely corrected each example in one step, then we would not find such a balance

Value function approximation by stochastic gradient descent

Stochastic gradient-descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \Big] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

The convergence results for SGD methods assume that α decreases over time

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty \qquad \left(\alpha_t = \frac{1}{t} \right)$$

Value function approximation by stochastic gradient descent

Stochastic gradient-descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \Big] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

The convergence results for SGD methods assume that α decreases over time

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty \qquad \left(\alpha_t = \frac{1}{t} \right)$$

Convergence to a local optimum

Features vector

- Represent state by a *feature vector*

$$\mathbf{x}(S) = \begin{pmatrix} \mathbf{x}_1(S) \\ \vdots \\ \mathbf{x}_n(S) \end{pmatrix}$$

- For example:
  - Distance of robot from landmarks
  - Trends in the stock market
  - Piece and pawn configurations in chess

Linear Value Function Approximation

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^{n} \mathbf{x}_j(S) \mathbf{w}_j$$

Linear Value Function Approximation

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^{n} \mathbf{x}_j(S) \mathbf{w}_j$$

- Objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w})^2 \right]$$

Linear Value Function Approximation

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^{n} \mathbf{x}_j(S) \mathbf{w}_j$$

- Objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w})^2 \right]$$

- Stochastic gradient descent converges on *global* optimum

Linear Value Function Approximation

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^{n} \mathbf{x}_j(S) \mathbf{w}_j$$

- Objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w})^2 \right]$$

- Stochastic gradient descent converges on *global* optimum
- Update rule is particularly simple

$$\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) = \mathbf{x}(S)$$

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \mathbf{x}(S)$$

Update = *step-size* $\times$ *prediction error* $\times$ *feature value*

Table Lookup Features

- Table lookup is a special case of linear value function approximation
- Using *table lookup features*

$$\mathbf{x}^{table}(S) = \begin{pmatrix} \mathbf{1}(S = s_1) \\ \vdots \\ \mathbf{1}(S = s_n) \end{pmatrix}$$

- Parameter vector $\mathbf{w}$ gives value of each individual state

$$\hat{v}(S, \mathbf{w}) = \begin{pmatrix} \mathbf{1}(S = s_1) \\ \vdots \\ \mathbf{1}(S = s_n) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{w}_1 \\ \vdots \\ \mathbf{w}_n \end{pmatrix}$$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$
  - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$
  - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

  - For TD(0), the target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$
  - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

  - For TD(0), the target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

  - For TD(λ), the target is the λ-return G_t^λ

$$\Delta \mathbf{w} = \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Monte – Carlo with Value Function Approximation

- Return G_t is an unbiased, noisy sample of true value $v_\pi(S_t)$
- Can therefore apply supervised learning to "training data":

$$\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, ..., \langle S_T, G_T \rangle$$

- For example, using *linear Monte-Carlo policy evaluation*

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\mathbf{x}(S_t)\end{aligned}$$

- Monte-Carlo evaluation converges to a local optimum
- Even when using non-linear value function approximation

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input : π, $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \ldots, T-1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G_t - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input : π, $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \ldots, T-1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G_t - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$$

Remember that $\mathbb{E}[G_t | S_t = s] = v_\pi(S_t)$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input : π, $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \ldots, T-1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G_t - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$$

Remember that $\mathbb{E}[G_t | S_t = s] = v_\pi(S_t)$

$\Rightarrow$ MC target G_t is by definition an unbiased estimate of $v_\pi(S_t)$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input : π, $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \ldots, T-1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G_t - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$$

Remember that $\mathbb{E}[G_t | S_t = s] = v_\pi(S_t)$

$\Rightarrow$ MC target G_t is by definition an unbiased estimate of $v_\pi(S_t)$

$\Rightarrow$ SGD converges to a locally optimal approximation of $v_\pi(S_t)$ assuming

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

TD(0) Learning with Value Function Approximation

- The TD-target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ is a *biased* sample of true value $v_\pi(S_t)$
- Can still apply supervised learning to "training data":

$$\langle S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}) \rangle, ..., \langle S_{T-1}, R_T \rangle$$

- For example, using *linear TD(0)*

$$\begin{aligned} \Delta \mathbf{w} &= \alpha(R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) \\ &= \alpha \delta \mathbf{x}(S) \end{aligned}$$

- Linear TD(0) converges (close) to global optimum

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

Target does not depend on w

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha(v_\pi(S) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

Target does not depend on w

$$\Delta \mathbf{w} = \alpha(R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

It depends on w (a typical property of all bootstrapping methods)

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha(v_\pi(S) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

Target does not depend on w

$$\Delta \mathbf{w} = \alpha(R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

It depends on w (a typical property of all bootstrapping methods)

They are not true gradient descent method: *they take into account the effect of changing the weight vector w_t on the estimate but ignore its effect on the target*

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha(v_\pi(S) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

Target does not depend on w

$$\Delta \mathbf{w} = \alpha(R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}))\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

It depends on w (a typical property of all bootstrapping methods)

They are not true gradient descent method: *they take into account the effect of changing the weight vector w_t on the estimate but ignore its effect on the target*

They are **semi-gradient methods**

TD(0) Learning with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: π, $\hat{v}: \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode):

Initialize S

Repeat (for each step of the episode):

Choose $A \sim \pi(\cdot|S)$

Take action A, observe R, S'

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\right] \nabla \hat{v}(S, \mathbf{w})$$

$S \leftarrow S'$

Until S' is terminal

TD(0) Learning with Value Function Approximation

- Since bootstrapping targets are biased, they do *not have the general convergence properties of the MC targets*
- Typically they enable *online, continual and faster learning* (they do not wait for the end of the episode)
- The semi-gradient TD(0) algorithm presented in the previous slide *converge reliably in important cases as the linear case*; this does not follow from general results on SGD but a separate theorem is necessary.
- The weight vector converged to is *not the global optimum*, but rather a point near the local optimum.

TD(0) Learning with Linear Value Function Approximation

- *The semi-gradient TD(0) algorithm presented in the previous slide also converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.*

- Once the system has reached steady state, the expected next weight vector can be written

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b}$$

- *The weight vector converged to is also not the global optimum, but rather a point near the local optimum*

n-step Learning with Value Function Approximation

n-step semi-gradient TD for estimating $\hat{v} \approx v_\pi$

Input: π, $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ $(\hat{v}(\text{terminal},\cdot) = 0)$, step-size α, integer n

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode):

Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \ldots, T-1$):

$$G \leftarrow \sum_{i=t+1}^{\min(t+n,T)} \gamma^{(i-t-1)} R_i$$

If $t+n < T$, then: $G \leftarrow G + \gamma^n \hat{v}(S_{t+n}, \mathbf{w})$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$$

Material

The material illustrated today can be found in the first four paragraphs of Chapter 9.

In the book there are more descriptions and thoughs. For the final examination pay attention to what seen in the slides used for the lecture (as usual concentrate on the algorithms!)